Java Collections Framework — Complete Guide

Bhai, tumhe basic Java aata hai to ye guide seedha collections pe focus karega — kya hai, kaise define karte hai, aur insert/remove/search kaise karte hai. Sab kuch code examples ke saath.

1. Collection Framework ka Structure

```
Collection (interface)
├── List   → ordered, duplicates allowed
├── Set    → unique elements
└── Queue  → FIFO / priority based
    └── Deque → dono taraf se insert/remove

Map (alag hierarchy — key-value pairs)
```

Sabse common jo use hote hai: `ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `HashMap`, `TreeMap`, `PriorityQueue`, `ArrayDeque`.

Sab collections `java.util.*` package me hai, so top pe likho:

```java
import java.util.*;
```

2. List — ordered, index-based, duplicates allowed

ArrayList (most used — dynamic array)

```java
```

```
// Define
List<Integer> list = new ArrayList<>();

// Insert
list.add(10);       // end me add
list.add(0, 5);     // index 0 pe insert

// Access/Check
int x = list.get(0);        // get by index
boolean has = list.contains(10); // check present hai ya nahi
int size = list.size();

// Update
list.set(0, 100);     // index 0 ka value change

// Remove (pop jaisa)
list.remove(0);          // index se remove
list.remove(Integer.valueOf(10)); // VALUE se remove (careful: int vs Integer)

// Iterate
for (int val : list) {
    System.out.println(val);
}
```

⚠️ Important gotcha: `list.remove(1)` index 1 remove karega, but `list.remove(Integer.valueOf(1))` value 1 remove karega. Ye ek common confusion hai jab List<Integer> use karte ho.

LinkedList (List + Deque dono implement karta hai)

```
java
LinkedList<Integer> ll = new LinkedList<>();
ll.addFirst(1);  // front me insert
ll.addLast(2);   // end me insert
ll.removeFirst(); // front se pop
ll.removeLast();  // end se pop
ll.peekFirst();  // dekho bina remove kiye
```

Use ArrayList jab random access chahiye ho (get by index fast), LinkedList jab bar bar insert/delete front/middle me karna ho.

3. Set — sirf unique elements, no duplicates

HashSet (fastest, no order guarantee)

```
java
```

```
Set<Integer> set = new HashSet<>();
set.add(10);
set.add(20);
set.add(10);       // duplicate — ignore ho jayega

boolean has = set.contains(10);  // check
set.remove(10);                  // remove

for (int val : set) { ... }     // order guarantee nahi
```

LinkedHashSet (insertion order maintain karta hai)

```
java
Set<Integer> lhs = new LinkedHashSet<>();
```

TreeSet (sorted order me rakhta hai)

```
java
Set<Integer> ts = new TreeSet<>();
ts.add(30);
ts.add(10);
ts.add(20);
// print karoge to sorted order aayega: 10, 20, 30

TreeSet<Integer> t = (TreeSet<Integer>) ts;
t.first();  // sabse chhota
t.last();   // sabse bada
t.higher(10); // 10 se bada next element
t.lower(20);  // 20 se chhota previous element
```

4. Map — Key-Value pairs

HashMap (most used)

```
java
```

```java
Map<String, Integer> map = new HashMap<>();

// Insert
map.put("apple", 10);
map.put("banana", 20);

// Update (same key dobara put karo to overwrite)
map.put("apple", 15);

// Access
int val = map.get("apple");      // 15
int def = map.getOrDefault("mango", 0); // key nahi hai to default value

// Check
boolean hasKey = map.containsKey("apple");
boolean hasVal = map.containsValue(20);

// Remove
map.remove("apple");

// Iterate
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
}

// Sirf keys / sirf values
for (String key : map.keySet()) { ... }
for (int val : map.values()) { ... }

// Common pattern: counting frequency
map.put(key, map.getOrDefault(key, 0) + 1);
```

LinkedHashMap — insertion order maintain

TreeMap — key ke hisaab se sorted

```java
Map<String, Integer> tm = new TreeMap<>();
// keys automatically sorted order me rahenge
```

5. Queue & Deque — FIFO / priority based

Queue (interface, LinkedList se implement)

```java
```

```java
Queue<Integer> q = new LinkedList<>();
q.offer(10);    // insert (add bhi chalega)
q.offer(20);
int front = q.peek();  // front dekho bina remove kiye
int removed = q.poll(); // front se pop (remove + return)
```

ArrayDeque (dono taraf se insert/pop — stack ya queue dono ke liye use hota hai)

```java
Deque<Integer> dq = new ArrayDeque<>();

// as Stack (LIFO)
dq.push(10);  // top pe insert
dq.push(20);
int top = dq.peek();  // top dekho
int popped = dq.pop(); // top se pop

// as Queue (FIFO)
dq.offerLast(10);  // end me insert
int f = dq.pollFirst(); // front se remove
```

PriorityQueue (hamesha smallest/largest element top pe rakhta hai)

```java
// Min-Heap (default — sabse chhota top pe)
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);
int min = pq.poll();  // 10 nikalega

// Max-Heap
PriorityQueue<Integer> maxPq = new PriorityQueue<>(Collections.reverseOrder());
```

Stack (purana class, but common hai)

```java
Stack<Integer> st = new Stack<>();
st.push(10);
st.push(20);
int top = st.peek();
int popped = st.pop();
boolean empty = st.isEmpty();
```

6. Quick Cheat-Sheet: kab kya use karo

Chahiye kya	Use karo
Index se access, order matters, duplicates ok	`ArrayList`
Bar bar insert/delete beech me	`LinkedList`
Sirf unique values	`HashSet`
Unique + insertion order	`LinkedHashSet`
Unique + sorted order	`TreeSet`
Key-value, fast access	`HashMap`
Key-value, sorted by key	`TreeMap`
Stack (LIFO)	`ArrayDeque` (push/pop)
Queue (FIFO)	`ArrayDeque` (offer/poll)
Smallest/largest jaldi chahiye	`PriorityQueue`

7. Sorting collections

```java
// List sort
List<Integer> l = new ArrayList<>(List.of(5, 2, 8, 1));
Collections.sort(l);              // ascending
Collections.sort(l, Collections.reverseOrder()); // descending

// Custom sort (objects)
list.sort((a, b) -> a.age - b.age);  // lambda comparator
```

8. Ek chhota practice example (sab mila ke)

```java
```

```java
import java.util.*;

public class Test {
  public static void main(String[] args) {
    // Frequency count using HashMap
    int[] arr = {1, 2, 2, 3, 3, 3};
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : arr) {
      freq.put(num, freq.getOrDefault(num, 0) + 1);
    }
    System.out.println(freq); // {1=1, 2=2, 3=3}

    // Stack se check balanced brackets jaisa kaam
    Deque<Character> stack = new ArrayDeque<>();
    stack.push('(');
    System.out.println(stack.pop()); // (

    // TreeSet se sorted unique values
    Set<Integer> unique = new TreeSet<>(Arrays.asList(5, 1, 3, 1, 5));
    System.out.println(unique); // [1, 3, 5]
  }
}
```

Bhai isko practice karne ke liye sabse best tarika: LeetCode/GFG pe easy problems try karo jisme "count frequency", "remove duplicates", "check balanced parentheses" jaisa kaam ho — yahi 4-5 collections repeat repeat use hote rahenge automatically yaad ho jayenge.